**Prolog Programming Assignment: Minimax Adversarial Search (Tic-Tac-Toe)**

All members contributed in equal measure

In completing this assignment, all team members have followed the honor pledge specified by the instructor for this course.

1. **Instrumentation Results**

   We instrumented our program using:

   - clear_count/0
   - inc_count/0
   - get_count/1

   The counter increments once per evaluated board inside minimax_value/3.

   Below are the results for three test positions.

   **Position 1 – Empty Board**

   Board: [e,e,e,
   e,e,e,
   e,e,e]

   Query used:

   ```
   clear_count.
   minimax_value([e,e,e,e,e,e,e,e,e], x, V).
   get_count(N).
   ```

   Result:

   - Minimax Value: V = 0
   - Board Expansions: N = 549946

   **Position 2 – Mid-Game Position**

Board: [x,o,e,

e,x,e,

o,e,e]

Query used:

```
clear_count.
minimax_value([x,o,e,e,x,e,o,e,e], x, V).
get_count(N).
```

Result:

- Minimax Value: V = 1
- Board Expansions: N = 182

**Position 3 – Near-End Position**

Board: [x,o,x,

x,o,e,

o,x,e]

Query used:

```
clear_count.
minimax_value([x,o,x,x,o,e,o,x,e], x, V).
get_count(N).
```

Result:

- Minimax Value: V = 0
- Board Expansions: N = 5

2. **Written Questions**

**Question 1- Why is minimax optimal under the assumptions (deterministic, perfect information, zero-sum)?**

Minimax is optimal because the game is deterministic, meaning there is no randomness in state transitions. It is also a perfect-information game, so both players can see the entire board at all times. Because the game is zero-sum, one player's gain is the other player's loss. Under these assumptions, minimax explores all possible future game states and assumes that the opponent will always play optimally to minimize our score. Therefore, it guarantees the best possible outcome against a rational opponent.

**Question 2 - What does a minimax value mean "in words"?**

A minimax value represents the final game outcome that will occur if both players play optimally from the current position. A value of 1 means that player X can force a win, while a value of -1 means that player O can force a win, and finally, a value of 0 means the game will end in a draw, assuming perfect play from both players.

**Question 3 - Why is minimax infeasible for large games like chess?**

Minimax becomes infeasible for large games because the number of possible game states grows exponentially as the number of possible moves increases. Chess has a very large branching factor and many possible move sequences, leading to an enormous game tree. Searching the entire tree would require unrealistic amounts of time and memory. As a result, practical chess programs use optimizations such as depth limits, heuristic evaluation functions, and alpha-beta pruning to reduce the number of states explored.